

BEGINNER'S GUIDE

Git & GitHub

A complete, friendly introduction — from your first commit to collaborating with pull requests.

12	90+	0
Chapters	Commands	Prior experience needed

Table of Contents

1	**What are Git & GitHub?**	Version control, history, why it matters
2	**Installing & Configuring Git**	Setup, identity, editor, first-time config
3	**Your First Repository**	init, status, add, commit
4	**The Git Workflow**	Working directory, staging area, repository
5	**Branching & Merging**	Creating, switching, merging, conflicts
6	**Connecting to GitHub**	Remotes, clone, push, pull, SSH keys
7	**Viewing History**	log, diff, blame, show
8	**Undoing Changes**	checkout, reset, revert, stash
9	**.gitignore & Best Practices**	Ignoring files, commit hygiene
10	**Collaborating on GitHub**	Fork, pull requests, code review, issues
11	**GitHub Extras**	Actions, Pages, Releases, README
12	**Cheat Sheet**	Everything on one page

CHAPTER 01

What are Git & GitHub?

Understanding version control, and why every developer uses it

Git is a free, open-source **distributed version control system** created by Linus Torvalds in 2005 (the same person who created the Linux kernel). It tracks changes to files over time, so you can see exactly what changed, who changed it, and roll back to any earlier point whenever you need to.

GitHub is a completely different thing: a **website and cloud service** that hosts Git repositories online. It adds collaboration features on top of Git — pull requests, issues, code review, project boards, and CI/CD pipelines. Think of it this way:

> **Simple analogy:** Git is like the engine that tracks your document's history on your own computer. GitHub is like Google Drive for that engine — a place to store it online, share it, and work on it with a team.

Why Use Version Control?

- **Full history** — every change is saved forever, with who made it and why.
- **Safe experimentation** — try things on a branch without breaking the main code.
- **Undo anything** — roll back a bad change in seconds.
- **Team collaboration** — many people can work on the same project without overwriting each other.
- **Backup** — your code lives safely on GitHub's servers, not just your laptop.
- **Industry standard** — used by virtually every software team in the world.

Git vs. GitHub vs. Other Platforms

Tool	What it is	Runs where
Git	Version control software	Your local computer
GitHub	Hosting + collaboration platform for Git repos	github.com (cloud)
GitLab	Alternative to GitHub, similar features	gitlab.com or self-hosted
Bitbucket	Alternative by Atlassian, integrates with Jira	bitbucket.org

Key Vocabulary

Term	Meaning
Repository (repo)	A project folder tracked by Git
Commit	A saved snapshot of your changes
Branch	An independent line of development
Remote	A version of your repo hosted elsewhere (e.g. GitHub)
Clone	A local copy of a remote repository
Pull Request (PR)	A request to merge your changes into another branch

Good to know: You don't need to be online to use Git — commits, branches, and history all work fully offline. You only need internet when pushing to or pulling from GitHub.

CHAPTER 02

Installing & Configuring Git

Get Git running and tell it who you are

Installing Git

OS	Command / Method
Ubuntu / Debian	`sudo apt install git`
Fedora / RHEL	`sudo dnf install git`
Arch Linux	`sudo pacman -S git`
macOS	`brew install git` (or Xcode Command Line Tools)
Windows	Download "Git for Windows" from git-scm.com

```
git --version      # confirm installation
→ git version 2.43.0
```

First-Time Setup

Before your first commit, tell Git your name and email — this information is attached to every commit you make.

```
git config --global user.name "Nagesh"
git config --global user.email "nagesh@example.com"

# Check your settings
git config --list
```

Useful Global Settings

```
git config --global init.defaultBranch main    # use "main" instead of "master"
git config --global core.editor "nano"           # set default commit-message editor
git config --global color.ui auto                  # enable colorful output
```

Where Settings Live

Scope	Flag	File location
System-wide	`--system`	`/etc/gitconfig`
Per user	`--global`	`~/.gitconfig`
Per repository	(none)	`.git/config`

> **Tip:** Create a free account at `github.com` now if you haven't already — you'll connect to it in Chapter 6.

CHAPTER 03

Your First Repository

Turning a folder into a Git-tracked project

Creating a Repository

```
mkdir my-project && cd my-project
git init
→ Initialized empty Git repository in /home/nagesh/my-project/.git/
```

This creates a hidden `.git` folder that stores the entire history of your project. Deleting it removes all Git tracking (but not your files).

Checking Status

```
git status
→ On branch main
   No commits yet
   nothing to commit (create/copy files and use "git add" to track)
```

Making Your First Commit

```
echo "# My Project" > README.md
git status
→ Untracked files: README.md

git add README.md              # stage the file
git commit -m "Initial commit"   # save a snapshot
→ [main (root-commit) a1b2c3d] Initial commit
    1 file changed, 1 insertion(+)
```

Staging Multiple Files

```
git add file1.txt file2.txt    # stage specific files
git add .                        # stage everything in the current folder
git add -A                         # stage everything, including deletions
```

Writing Good Commit Messages

✓ Good

```
git commit -m "Fix login button alignment on mobile"
```

✗ Avoid

```
git commit -m "fixed stuff"
```

Convention: Use the imperative mood — "Add feature" not "Added feature" — and keep the first line under ~50 characters. Add more detail on following lines if needed.

CHAPTER 04

The Git Workflow

Three areas every file moves through

Every file in a Git repository lives in one of three places. Understanding this flow is the single most important concept in Git.

📁 **Working Directory**
Your actual files, as you edit them on disk

→

📋 **Staging Area**
Changes you've marked ready with `git add`

→

🗄️ **Repository**
Permanent snapshots saved with `git commit`

Moving Through the Stages

```
nano app.py                # 1. Edit a file (Working Directory)
git add app.py               # 2. Stage the change (Staging Area)
git commit -m "Update app"     # 3. Save permanently (Repository)
```

Checking Where Things Stand

```
git status          # see what's staged, unstaged, and untracked
git diff              # see unstaged changes (working dir vs staging)
git diff --staged       # see staged changes (staging vs last commit)
```

File States

State	Meaning
Untracked	Git doesn't know about this file yet
Modified	File changed but not staged
Staged	Marked to be included in the next commit
Committed	Safely saved in the repository's history

Shortcut: `git commit -am "message"` stages and commits all *already-tracked, modified* files in one step (it won't pick up brand-new untracked files).

CHAPTER 05

Branching & Merging

Work on features safely, in parallel, without breaking main

A **branch** is an independent line of development. The default branch is usually called `main`. Creating a branch lets you experiment freely — nothing affects `main` until you merge.



Creating & Switching Branches

```
git branch                      # list all branches
git branch feature-login          # create a new branch
git switch feature-login            # switch to it
git switch -c feature-signup          # create AND switch in one step
git checkout -c feature-signup         # older equivalent of switch -c
```

Making Changes on a Branch

```
git switch feature-login
nano login.py
git add login.py
git commit -m "Add login form validation"
```

Merging Back into Main

```
git switch main                   # go to the branch you want to merge INTO
git merge feature-login              # bring feature-login's changes into main
→ Fast-forward
   login.py | 12 ++++++++++++
```

Deleting a Branch After Merging

```
git branch -d feature-login       # delete (safe — warns if unmerged)
git branch -D feature-login         # force delete, even if unmerged
```

Resolving Merge Conflicts

A conflict happens when two branches change the same lines. Git pauses and marks the conflict directly in the file:

```
<<<<<<< HEAD
print("Hello from main")
=======
print("Hello from feature-login")
>>>>>>> feature-login
```

```
# Edit the file to keep the correct code, then:
git add login.py
git commit -m "Resolve merge conflict in login.py"
```

Beginner tip: Merge conflicts look scary but are normal. Just open the file, decide which lines to keep (or combine both), delete the `<<<<<<<` / `=======` / `>>>>>>>` markers, then add and commit.

CHAPTER 06

Connecting to GitHub

Remotes, cloning, pushing, and pulling

Two Ways to Start

Start on GitHub, clone locally

```
git clone https://github.com/nagesh/my-project.git
cd my-project
```

Start locally, push to GitHub

```
git remote add origin https://github.com/nagesh/my-project.git
git push -u origin main
```

Understanding Remotes

```
git remote -v
→ origin  https://github.com/nagesh/my-project.git (fetch)
   origin  https://github.com/nagesh/my-project.git (push)
```

`origin` is just a nickname for the remote URL — the default name Git gives the first remote you add.

Pushing & Pulling

```
git push                    # send local commits to GitHub
git push origin main          # push a specific branch explicitly
git pull                        # fetch + merge remote changes into your branch
git fetch                         # download remote changes WITHOUT merging
```

Authenticating with GitHub (SSH)

HTTPS works with a username/token, but SSH keys are the smoother long-term option:

```
ssh-keygen -t ed25519 -C "nagesh@example.com"   # generate a key pair
cat ~/.ssh/id_ed25519.pub                            # copy this into GitHub → Settings → SSH Keys
ssh -T git@github.com                                  # test the connection
→ Hi nagesh! You've successfully authenticated.
```

```
git remote set-url origin git@github.com:nagesh/my-project.git   # switch remote to SSH
```

Cloning vs. Forking

Action	What happens
`git clone`	Downloads a copy of a repo to your computer
Fork (on GitHub)	Creates your own copy of someone else's repo *on GitHub*

> **Typical team flow:** Fork a repo on GitHub → clone your fork locally → make changes on a branch → push to your fork → open a Pull Request back to the original repo.

CHAPTER 07

Viewing History

Exploring what changed, when, and by whom

The Commit Log

```
git log
→ commit a1b2c3d4e5f6...
   Author: Nagesh <nagesh@example.com>
   Date:   Wed Sep 9 10:32:04 2026 +0530

       Add login form validation
```

Friendlier Log Formats

```
git log --oneline                    # one line per commit
git log --oneline --graph --all        # visual branch graph
git log -5                               # show only the last 5 commits
git log --author="Nagesh"                  # filter by author
git log --since="2 weeks ago"                # filter by date
```

```
git log --oneline --graph --all
→ * a1b2c3d (HEAD -> main) Merge feature-login
→ |\
→ | * 9f8e7d6 Add login validation
→ |/
→ * 5c4b3a2 Initial commit
```

Inspecting Changes

```
git diff                        # unstaged changes
git diff HEAD~1 HEAD              # compare two commits
git show a1b2c3d                    # full details of one commit
git blame login.py                    # who last changed each line, and when
```

Referring to Commits

Reference	Meaning
`HEAD`	The commit you currently have checked out
`HEAD~1`	One commit before HEAD
`HEAD~3`	Three commits before HEAD
`a1b2c3d`	A specific commit, by its short hash

CHAPTER 08

Undoing Changes

Every developer's most-searched Git topic

Discarding Uncommitted Changes

```
git restore login.py            # discard unstaged changes in one file
git restore .                      # discard ALL unstaged changes
git restore --staged login.py        # unstage a file (keep the edits)
```

Amending the Last Commit

```
git commit --amend -m "Corrected commit message"    # fix the message
git add forgotten_file.py
git commit --amend --no-edit                            # add a forgotten file, same message
```

Reset vs. Revert

`git reset` — rewrites history, moves the branch pointer backward. Best for *local, unpushed* commits.

```
git reset --soft HEAD~1    # undo commit, keep changes staged
git reset --mixed HEAD~1     # undo commit, unstage changes
git reset --hard HEAD~1        # undo commit, DELETE changes
```

`git revert` — creates a *new* commit that undoes a previous one. Safe for commits already pushed/ shared.

```
git revert a1b2c3d
→ Creates a new commit reversing a1b2c3d
```

Careful: `git reset --hard` permanently discards changes with no undo. Never use `reset` on commits that others have already pulled — use `revert` instead to avoid rewriting shared history.

Stashing Work in Progress

Need to switch branches but aren't ready to commit? Stash your changes temporarily.

```
git stash                    # save uncommitted changes, clean working directory
git stash list                 # see all stashes
git stash pop                    # reapply the most recent stash and remove it
git stash apply                    # reapply without removing from the stash list
git stash drop                       # delete a stash without applying it
```

CHAPTER 09

.gitignore & Best Practices

Keeping your repository clean and secure

A `.gitignore` file tells Git which files or folders to never track — things like build output, dependencies, and secrets that shouldn't be shared.

Example .gitignore

```
# .gitignore
node_modules/          # dependency folders
*.log                    # all log files
.env                       # environment variables / secrets
__pycache__/                  # Python cache
.DS_Store                       # macOS system file
dist/                              # build output
*.pyc
```

Pattern Syntax

Pattern	Matches
`*.log`	Any file ending in .log, anywhere
`build/`	A folder named "build", anywhere
`/config.json`	Only config.json in the root folder
`!important.log`	Exception — do track this one file

If a File Is Already Tracked

Adding a pattern to `.gitignore` only affects *untracked* files. To stop tracking something already committed:

```
git rm --cached secrets.env
git commit -m "Stop tracking secrets.env"
```

Commit Hygiene Best Practices

Do

- Commit small, focused changes
- Write clear, descriptive messages
- Pull before you start new work
- Use branches for features/fixes

Avoid

- Committing secrets or API keys
- Giant commits mixing unrelated changes
- Committing directly to `main` on teams
- Force-pushing to shared branches

Leaked a secret? Removing it in a new commit isn't enough — it still exists in history. Rotate/revoke the credential immediately, then consider tools like `git filter-repo` to scrub history.

CHAPTER 10

Collaborating on GitHub

Forks, pull requests, reviews, and issues

The Fork & Pull Request Workflow

1. Fork	→	2. Branch	→	3. Commit	→	4. PR
Copy the repo to your own GitHub account		Create a feature branch in your fork		Make and push your changes		Open a Pull Request to the original repo

```
git clone https://github.com/nagesh/awesome-project.git
cd awesome-project
git switch -c fix-typo
# ...make changes...
git add .
git commit -m "Fix typo in README"
git push origin fix-typo
# → Open a Pull Request on github.com from this branch
```

Writing a Good Pull Request

- A clear title summarizing the change
- A description of *what* changed and *why*
- Screenshots for UI changes, if relevant
- A link to the related issue, e.g. `Closes #42`

Code Review

Action	What it means
Comment	Leave feedback without approving or blocking
Approve	Sign off — the PR is ready to merge
Request changes	Block merging until issues are addressed

Merging a Pull Request

Method	Result
Merge commit	Keeps full branch history, adds a merge commit
Squash and merge	Combines all commits into one clean commit
Rebase and merge	Replays commits on top, linear history, no merge commit

Issues

Issues track bugs, feature requests, and tasks. Reference them in commits or PRs to auto-link and even auto-close them:

```
git commit -m "Fix crash on empty input, closes #42"
```

> **Tip:** Use GitHub's issue and PR templates (in a `.github/` folder) to standardize how your team reports bugs and submits changes.

CHAPTER 11

GitHub Extras

Beyond code hosting: automation, docs, and releases

README.md

The first file visitors see on your repo. Written in Markdown, it typically covers what the project does, how to install it, and how to use it.

```
# My Project

A short description of what this does.

## Installation
\`\`\`bash
git clone https://github.com/nagesh/my-project.git
\`\`\`
```

GitHub Actions (CI/CD)

Actions automatically run tasks — tests, builds, deployments — whenever you push or open a PR. Defined in YAML files under `.github/workflows/`.

```yaml
# .github/workflows/test.yml
name: Run Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest
```

GitHub Pages

Free static-site hosting straight from a repository — great for documentation, portfolios, and project sites.

```
Settings → Pages → Source: main branch /docs folder → Save
→ Live at https://nagesh.github.io/my-project
```

Releases & Tags

```bash
git tag v1.0.0                      # create a lightweight tag
git tag -a v1.0.0 -m "First stable release"    # annotated tag with message
git push origin v1.0.0                # push the tag to GitHub
git push origin --tags                  # push all tags
```

On GitHub, turning a tag into a **Release** lets you attach compiled binaries, changelogs, and notes for that version.

Other Handy Features

Feature	What it's for
Projects	Kanban-style boards for tracking tasks
Discussions	Community Q&A separate from issues
Codespaces	Full cloud dev environment in the browser
Dependabot	Automated dependency & security updates

CHAPTER 12

Cheat Sheet

The essentials, all in one place for quick reference

Setup

`git init`	start a repo
`git clone url`	copy a repo
`git config`	set identity/prefs

Staging & Committing

`git status`	check state
`git add f`	stage file
`git commit -m ""`	save snapshot

Branching

`git branch`	list branches
`git switch -c b`	create & switch
`git merge b`	merge branch b

Remote / GitHub

`git remote -v`	show remotes
`git push`	upload commits
`git pull`	download + merge

History

`git log --oneline`	compact log
`git diff`	show changes
`git show hash`	one commit's detail

Undoing

`git restore f`	discard edits
`git reset --hard`	rewind + discard
`git revert hash`	undo safely

Stash

`git stash`	save WIP
`git stash pop`	reapply WIP
`git stash list`	list stashes

Tags & Releases

`git tag v1.0`	create tag
`git push --tags`	push tags

Typical Daily Flow

```
git pull                              # 1. get latest changes
git switch -c fix-navbar                # 2. create a branch
# ...edit files...
git add .                                  # 3. stage
git commit -m "Fix navbar spacing"           # 4. commit
git push origin fix-navbar                     # 5. push
# 6. open a Pull Request on GitHub
```

You've now covered the core of everyday Git & GitHub use. The best next step is practice — init a repo, make a mess, and undo it. Happy committing!